FinBank 완전 가이드

수신부터 여신·배치·마감까지 — 설계·코드·DB·테스트 전부 한 권으로

2026-07-28

Contents

0. 이 문서를 읽는 법 **4**

1. FinBank가 뭔가요? **4**
1-1. 왜 은행인가? . . . 4
1-2. 무엇으로 만들었나 (기술 목록) . . . 4

2. 은행 업무를 둘로 나누기: 수신과 여신 **5**
2-1. 수신(受信) — 맡아주는 돈 . . . 5
2-2. 여신(與信) — 빌려주는 돈 . . . 5
2-3. 이 프로젝트가 걸어온 순서 . . . 5

3. 데이터베이스 전략 — 표를 어떻게 만들었나 **6**
3-1. 데이터베이스가 뭔가요? . . . 6
3-2. 전체 표 목록 (10개) . . . 6
3-3. 표들이 어떻게 연결되나 . . . 6
3-4. 설계에서 가장 중요한 결정 4가지 . . . 7
3-5. Flyway — DB 변경 이력 관리 . . . 7

4. 패키지(폴더) 구성 — 코드를 어디에 두었나 **9**
4-1. 전체 구조 . . . 9
4-2. 왜 이렇게 나눴나 — “도메인형” 구조 . . . 9
4-3. 의존 방향 — 화살표는 한쪽으로만 . . . 10

5. 수신 파트 — 맡아주는 돈 **11**
5-1. 로그인은 어떻게 되나 (JWT) . . . 11
5-2. 계좌 만들기 . . . 11
5-3. 입금·출금·이체 . . . 11
5-4. 동시성 — 이 프로젝트의 심장 . . . 12

6. 돈을 다루는 규칙 — BigDecimal 이야기 **14**
6-1. 왜 소수가 필요한가 . . . 14
6-2. 해결: BigDecimal + DECIMAL(19,4) . . . 14
6-3. 전환 시점을 왜 그때로 잡았나 . . . 14
6-4. 규칙을 한 곳에 모았다 — `MoneyPolicy` . . . 14

7. 여신 파트 — 빌려주는 돈 **15**
7-1. 대출 상품 (`loan_products`) . . . 15
7-2. 상환 방식 3가지 — 그림으로 이해하기 . . . 15
7-3. 끝수 조정 — 1원이 남는 문제 . . . 15
7-4. 검증을 3층으로 했다 . . . 16
7-5. 자동심사 — 룰을 클래스 하나씩 . . . 16
7-6. 대출 실행 — 돈이 실제로 나가는 순간 . . . 17

7-7. 상환 — 갚기 . . . 17

8. 멱등키 — "두 번 눌러도 한 번만" 18
8-1. 문제 상황 . . . 18
8-2. 해결: 요청마다 고유 번호표 . . . 18
8-3. 화면에서 키를 어떻게 다뤄야 하나 . . . 18
8-4. 트랜잭션 경계 — 가장 어려운 부분 . . . 18
8-5. 경쟁의 심판은 DB에게 맡긴다 . . . 18
8-6. 실제 측정 결과 . . . 19
8-7. 이중 처리 방어 — 층마다 다른 걸 막는다 . . . 19
8-8. 대출 상환의 락 순서 . . . 19

9. 검색과 인덱스 — 느린 걸 빠르게 20
9-1. 문제: 조건이 7개인 검색 . . . 20
9-2. 해결: QueryDSL . . . 20
9-3. 인덱스 — 책의 목차 . . . 20
9-4. 실제로 재봤다 (10만 건) . . . 20
9-5. 인덱스 컬럼 순서가 중요하다 . . . 20
9-6. 정직하게 기록한 한계 . . . 20

10. 배치 — 밤에 혼자 도는 프로그램 21
10-1. 배치가 뭔가요? . . . 21
10-2. 가장 조심해야 할 함정: 페이징 밀림 . . . 21
10-3. 멱등을 세 겹으로 . . . 21
10-4. 연체이자 기산점 — 구간이 이어지게 . . . 21
10-5. 청크 사이즈 — 재보고 정했다 . . . 22
10-6. 스케줄러의 한계를 정직하게 적었다 . . . 22

11. 예외 처리 — 오류를 어떻게 알려주나 23
11-1. 상태 코드로 구분한다 . . . 23
11-2. ErrorCode로 한 곳에 모았다 . . . 23
11-3. 점진 이관 (expand-contract) . . . 23

12. 테스트 — 333개가 무엇을 지키나 24
12-1. 철학: 개수보다 "위험한 곳" . . . 24
12-2. 테스트 파일별 역할 (전체) . . . 24
12-3. 왜 진짜 MySQL로 테스트하나 . . . 25
12-4. 성능 테스트를 분리한 이유 . . . 25
12-5. 테스트 DB 분리 . . . 25

13. 자동화와 실행 환경 26
13-1. CI (GitHub Actions) — 자동 감독관 . . . 26
13-2. Docker Compose — 한 번에 켜기 . . . 26
13-3. 설정 우선순위 . . . 26

14. 실패 기록 8건 — 무엇이 왜 깨졌나 27
① BigDecimal 전환 후 테스트 전멸 . . . 27
② Flyway 유령 마이그레이션 . . . 27
③ 코드를 안 고쳤는데 어느 날 깨지는 테스트 . . . 27
④ 인덱스를 지울 수 없다 (MySQL 에러 1553) . . . 27
⑤ 배치 테스트가 두 번째부터 실패 . . . 27
⑥ 배치를 만들자 드러난 상환의 빈틈 . . . 27
⑦ PAT 권한 부족으로 push 거부 . . . 27
⑧ 테스트 설정 파일 하나 추가했더니 컨텍스트 전체가 죽었다 . . . 28

15. 면접 예상 질문과 답 29

16. 한 장 요약 31

이 프로젝트가 증명하는 것 . . . 31
전 구간 흐름 . . . 31
처음부터 다시 만든다면 — 순서 . . . 31

부록 A. 코드 상세 레퍼런스 **32**
A-1. 도메인 — 자기 규칙을 스스로 지키는 객체 . . . 32
A-2. 계산 엔진 — 스프링을 모르는 순수 자바 . . . 34
A-3. 리포지토리 — DB에 말 거는 층 . . . 34
A-4. 서비스 — 규칙이 사는 곳 . . . 35
A-5. 멱등 실행기 — 전체 코드 . . . 37
A-6. 컨트롤러 — 엔드포인트 전체 목록 . . . 37
A-7. 프론트엔드에서 멱등키 다루기 . . . 38

부록 B. 용어 사전 **40**

0. 이 문서를 읽는 법

이 문서는 **FinBank라는 프로젝트를 하나도 모르는 사람**이 처음부터 끝까지 읽으면 "아, 이 사람이 뭘 만들었고 왜 그렇게 만들었구나"를 알 수 있게 쓴 글입니다.

어려운 말이 나오면 반드시 그 자리에서 쉬운 말로 다시 설명합니다. 순서는 이렇습니다.

1. **FinBank가 뭔지** — 한 문장으로
2. **은행 업무를 둘로 나누기** — 수신(맡는 돈)과 여신(빌려주는 돈)
3. **데이터베이스 전략** — 표(테이블)를 어떻게 만들었고 왜 그렇게 만들었나
4. **폴더(패키지) 구성** — 코드를 어디에 두었나
5. **수신 파트** — 회원가입·로그인·계좌·입출금·이체, 그리고 **동시성**
6. **돈을 다루는 규칙** — 소수점과 반올림 이야기
7. **여신 파트** — 대출 상품·신청·심사·이자계산·실행·상환
8. **멱등키** — "두 번 눌러도 한 번만"
9. **검색과 인덱스** — 느린 걸 빠르게
10. **배치** — 밤에 혼자 도는 프로그램
11. **테스트** — 333개가 무엇을 지키고 있나
12. **자동화와 배포** — CI, Docker
13. **실패 기록 8건** — 무엇이 왜 깨졌고 어떻게 고쳤나
14. **면접 예상 질문과 답**

---

1. FinBank가 뭔가요?

FinBank는 "은행 서버"를 흉내 내서 만든 개인 프로젝트입니다.

화면(웹사이트)도 있지만, 진짜 핵심은 **눈에 보이지 않는 뒤쪽(백엔드)**입니다. 돈을 넣고, 빼고, 옮기고, 빌려주고, 갚는 모든 일이 여기서 벌어집니다.

1-1. 왜 은행인가?

프로그램을 만들 때 가장 어려운 문제 중 하나가 **"여러 사람이 같은 것을 동시에 만졌을 때 값이 틀어지는 것"**입니다. 그런데 은행은 이게 틀어지면 **바로 돈이 사라지거나 생깁니다.** 그래서 이 문제를 공부하기에 가장 좋은 소재입니다.

> **12살 버전**: 친구 다섯 명이 동시에 같은 저금통에 손을 넣어서 돈을 꺼낸다고 생각해 보세요. 저금통에 3천 원이 있는데 다섯 명이 각각 "1천 원 꺼낼게!"라고 동시에 외치면? 잘못 만들면 5천 원이 나와 버립니다. **없던 돈이 생기는 것**이죠. FinBank는 이런 일이 절대 일어나지 않게 막는 방법을 연구한 프로젝트입니다.

1-2. 무엇으로 만들었나 (기술 목록)

무엇	이름	쉬운 설명
언어	Java 17	프로그램을 쓰는 말
뼈대	Spring Boot 3	서버를 쉽게 만들게 도와주는 도구상자
데이터 저장소	MySQL 8	표(엑셀 비슷한 것)로 자료를 저장하는 창고
자료 다루기	JPA / Hibernate	자바 객체와 DB 표를 자동으로 연결해 주는 통역사
표 관리	Flyway	DB 표의 변경 이력을 순서대로 관리하는 공책
검색	QueryDSL	조건이 여러 개인 검색을 자바 코드로 안전하게 쓰는 도구
밤 작업	Spring Batch	대량 작업을 안전하게 돌리는 도구
로그인	JWT + Spring Security	출입증을 만들고 검사하는 장치
화면	React + Vite + Tailwind	사용자가 보는 웹 화면
자동 검사	GitHub Actions	코드를 올릴 때마다 자동으로 시험을 보는 감독관
실행 환경	Docker Compose	서버와 DB를 한 번에 켜는 스위치

---

2. 은행 업무를 둘로 나누기: 수신과 여신

은행이 하는 일은 크게 **두 가지**입니다. 이 프로젝트도 정확히 그 둘로 나뉩니다.

2-1. 수신(受信) — 맡아주는 돈

"손님이 맡긴 돈을 보관하고 옮겨주는 일"입니다.

- 통장 만들기 (계좌 개설)
- 돈 넣기 (입금)
- 돈 빼기 (출금)
- 다른 통장으로 보내기 (이체)

12살 버전: 저금통을 만들어 주고, 돈을 넣고 빼주고, 친구 저금통으로 옮겨주는 일이에요.

2-2. 여신(與信) — 빌려주는 돈

"손님에게 돈을 빌려주고 이자를 받아 돌려받는 일"입니다.

- 대출 상품 (어떤 조건으로 빌려주는지)
- 대출 신청과 심사 (빌려줘도 되는 사람인가?)
- 대출 실행 (실제로 돈을 줌)
- 상환 (매달 나눠서 갚음)
- 연체 (안 갚으면 이자가 더 붙음)

12살 버전: 친구에게 만 원을 빌려주고 "매달 천 원씩 열 달 동안 갚아. 대신 조금 더 얹어서 줘"라고 하는 거예요. 그 '조금 더'가 **이자**입니다.

2-3. 이 프로젝트가 걸어온 순서

```
A. 동시성 문제 증명        (돈이 증발하는 걸 눈으로 보여주기)
B. 금액 타입 전환          (정수 → 소수 가능한 타입으로)
C. 여신 만들기             (상품 → 신청/심사 → 이자엔진 → 실행/상환 → 검색)
D. 배치                    (밤마다 연체를 찾아내고 이자를 붙이기)
E. 마감                    (자동 검사, Docker, 문서 정리)
```

중요한 원칙 두 가지를 처음부터 끝까지 지켰습니다.

1. **깊이 우선** — 기능을 넓게 여러 개 얕게 만들지 않고, 은행의 핵심인 대출 하나를 깊게 팠습니다.
2. **기술은 필요할 때만** — "이거 없으면 뭐가 안 되는데?"에 한 문장으로 답할 수 없으면 안 씁니다. 그래서 QueryDSL은 조건 검색이 생긴 C4에서, Spring Batch는 연체 정산이 생긴 D에서 넣었습니다. Redis 같은 건 **필요를 증명할 수 없어서 일부러 안 넣었습니다.**

---

3. 데이터베이스 전략 — 표를 어떻게 만들었나

3-1. 데이터베이스가 뭔가요?

엑셀 표를 아주 많이, 아주 빠르게, 아주 안전하게 다루는 창고라고 생각하면 됩니다.

- **테이블(표)**: members(회원 표), accounts(계좌 표)처럼 한 종류의 자료를 담는 표
- **행(row)**: 표의 한 줄. 회원 한 명, 계좌 하나
- **컬럼(열)**: 표의 세로 칸. 이름, 잔액, 만든 날짜
- **PK(기본키)**: 그 줄을 콕 집어 가리키는 번호표. 학생 번호 같은 것
- **FK(외래키)**: 다른 표의 번호표를 가리키는 손가락. "이 계좌는 3번 회원 거예요"
- **UNIQUE(유일 제약)**: "이 칸에는 같은 값이 두 번 들어올 수 없다"는 규칙

3-2. 전체 표 목록 (10개)

FinBank의 표는 **수신 4개 + 여신 6개**입니다.

수신 쪽 (4개)

표 이름	무엇을 담나
members	회원 (이메일, 이름, 비밀번호)
refresh_tokens	로그인 연장용 티켓
accounts	계좌 (계좌번호, 잔액, 잠김 여부)
transaction_logs	입출금·이체 **원장**(모든 거래 기록)

여신 쪽 (6개)

표 이름	무엇을 담나
loan_products	대출 상품 (금리, 한도, 상환방식)
loan_applications	대출 신청서 (입금받을 계좌 지정 + 심사 결과 포함)
loan_accounts	실행된 대출 계약
repayment_schedules	상환 스케줄 (1회차, 2회차… 각각 한 줄)
loan_transactions	대출 **원장** (실행/상환/연체이자)
overdue_histories	연체 이력

여기에 **멱등키 표** idempotency_keys 하나와, Spring Batch가 자동으로 만드는 BATCH_* 표들이 더 있습니다.

3-3. 표들이 어떻게 연결되나

```
members (회원)
   ├──1:1── refresh_tokens (로그인 티켓)
   ├──1:N── accounts (계좌)  ──1:N── transaction_logs (입출금 원장)
   ├──1:N── loan_applications (대출 신청)
   │          └──1:1── loan_accounts (대출 계약)
   │                     ├──1:N── repayment_schedules (상환 회차)
   │                     │          └──1:N── overdue_histories (연체 이력)
   │                     └──1:N── loan_transactions (대출 원장)
   └──1:N── loan_accounts (직접 연결도 있음 — 조회 편의)

loan_products (상품) ──1:N── loan_applications
accounts (입출금 계좌) ──1:N── loan_accounts (연결계좌: 실행 시 입금 / 상환 시 출금)
```

> **1:N이 뭔가요?** "하나에 여럿"이라는 뜻입니다. 회원 한 명이 계좌를 여러 개 가질 수 있으니 members 1:N accounts입니다.

3-4. 설계에서 가장 중요한 결정 4가지

① 원장(元帳)은 지우지 않는다 — append-only

transaction_logs와 loan_transactions는 **한 번 쓰면 절대 수정하거나 지우지 않습니다.** 잘못됐으면 반대 방향 기록을 새로 추가합니다.

> **12살 버전**: 일기장에 잘못 썼다고 찢어내면 안 됩니다. "어제 쓴 건 틀렸어요, 사실은 이랬어요"라고 **새 줄에 적는 것**입니다. 그래야 나중에 무슨 일이 있었는지 전부 알 수 있습니다.

왜? 은행에서 "그때 진짜 얼마가 오갔나"를 나중에 증명할 수 없으면 큰일 나기 때문입니다.

② 거래 직후 잔액을 같이 적는다 — balance_after

거래 기록에 "이 거래 후 잔액이 얼마가 됐는지"를 **함께** 저장합니다.

왜? 계좌 표의 잔액이 이상해 보일 때, 원장을 처음부터 더해보면 어디서 틀어졌는지 찾을 수 있기 때문입니다. 검산용 안전장치입니다.

③ 대출 실행 시점의 조건을 "사진 찍어" 보관한다 — 스냅샷

loan_accounts 표에는 interest_rate(금리), overdue_extra_rate(연체가산율), repayment_type(상환방식)이 **다시 저장**되어 있습니다. 상품 표에도 있는데 왜 또 저장할까요?

은행이 내일 상품 금리를 바꿔도, 어제 계약한 사람의 조건은 절대 바뀌면 안 되기 때문입니다.

> **12살 버전**: 친구랑 "만 원 빌려주고 매달 100원 이자"라고 약속했는데, 다음 달에 내가 "이제 200원이야"라고 바꾸면 안 되잖아요. **약속한 순간의 조건을 종이에 적어두는 것**이 스냅샷입니다.

④ 규칙은 DB에도 새겨 넣는다 — 제약조건

프로그램 코드로도 막고, **DB 표 자체에도** 막습니다. 이중 잠금입니다.

제약	무엇을 막나
uq_loan_account_application	신청 1건은 **딱 한 번만** 실행 가능
uq_schedule_installment	같은 대출에 같은 회차 번호가 두 번 생기지 못함
uq_loan_tx_idem	같은 멱등키로 거래가 두 번 기록되지 못함
uq_idem_key	같은 멱등키가 두 번 선점되지 못함

왜 두 겹인가? 코드는 실수로 빠뜨릴 수 있지만, **DB 제약은 무슨 일이 있어도 뚫리지 않습니다.** 마지막 방어선입니다.

3-5. Flyway — DB 변경 이력 관리

DB 표를 바꿀 때 손으로 직접 고치면, 내 컴퓨터와 서버의 표 모양이 달라져서 사고가 납니다. 그래서 **변경을 SQL 파일로 번호를 매겨 저장**하고, 프로그램이 켜질 때 순서대로 적용합니다.

파일	무엇을 했나
V1__init_schema.sql	회원·계좌·거래원장 표 생성
V2__add_refresh_tokens.sql	로그인 연장 티켓 표 추가
V3__money_to_decimal.sql	**금액 타입을 정수에서 소수 가능 타입으로 전환**
V4__create_loan_tables.sql	여신 6개 표 생성
V5__create_idempotency_keys.sql	멱등키 표 + 중도상환수수료율 컬럼
V6__add_search_indexes.sql	검색 속도용 인덱스 2개

두 가지 철칙이 있습니다.

1. **이미 적용된 파일은 절대 수정하지 않는다.** 고칠 게 있으면 새 번호(V7)를 만든다.
2. **JPA는 `ddl-auto: validate`.** 즉 JPA가 표를 마음대로 만들거나 바꾸지 못하게 하고, **"코드와 표 모양이 다르면 시작을 거부"**하게만 시킵니다.

   왜 validate인가? `update`로 두면 JPA가 알아서 표를 바꿔주는데, **무엇이 언제 바뀌었는지 기록이 안 남습니다.** 실무에서는 금지입니다.

---

4. 패키지(폴더) 구성 — 코드를 어디에 두었나

4-1. 전체 구조

```
com.finbank.backend
  ├── account/     수신: 계좌·입출금·이체
  │    ├── controller/   AccountController
  │    ├── service/      AccountService
  │    ├── domain/       Account, TransactionLog, TransactionType
  │    ├── repository/   AccountRepository, TransactionLogRepository
  │    └── dto/          요청·응답 그릇
  │
  ├── auth/        인증: 회원가입·로그인·토큰
  │    ├── controller/   AuthController
  │    ├── service/      AuthService
  │    ├── security/     JwtTokenProvider, JwtAuthenticationFilter
  │    ├── domain/       RefreshToken
  │    └── dto/
  │
  ├── member/      회원
  │    ├── domain/       Member
  │    ├── repository/   MemberRepository
  │    └── service/      CurrentMemberProvider (로그인한 사람 찾기 — 단일 창구)
  │
  ├── loan/        여신: 이 프로젝트의 핵심
  │    ├── controller/   LoanProductController, LoanApplicationController, LoanController
  │    ├── service/      LoanApplicationService, LoanExecutionService/Processor,
  │    │                 LoanRepaymentService/Processor, LoanQueryService
  │    │    └── review/   자동심사 룰들 (LoanReviewer 인터페이스 + 룰 3개)
  │    ├── domain/       LoanProduct, LoanApplication, LoanAccount,
  │    │    │            RepaymentSchedule, LoanTransaction, OverdueHistory + enum들
  │    │    └── calculator/  이자·수수료 계산 엔진 (순수 자바)
  │    ├── batch/        연체 감지·연체이자 가산 배치
  │    ├── repository/   QueryDSL 커스텀 포함
  │    ├── dto/
  │    └── config/       계산기 빈 등록
  │
  ├── common/      공통
  │    ├── MoneyPolicy       돈 계산 규칙의 단일 출처
  │    ├── PageResponse      페이징 응답 그릇
  │    └── idempotency/      멱등키 장치
  │
  ├── config/      SecurityConfig, SwaggerConfig, QuerydslConfig
  └── exception/   ErrorCode, ApiError, 예외 3종, GlobalExceptionHandler
```

4-2. 왜 이렇게 나눴나 — “도메인형” 구조

처음에는 controller/, service/, domain/처럼 **역할별로** 나눴습니다. 그런데 여신이 들어오면서 바꿨습니다.

- **역할별(계층형)**: service/ 폴더 안에 계좌·인증·대출 서비스가 전부 섞임
- **도메인형(지금)**: loan/ 폴더만 열면 대출에 관한 **모든 것**이 다 있음

  12살 버전: 옷장을 정리할 때 “윗도리 서랍/아랫도리 서랍”으로 나누는 게 계층형이고, “여름 옷 칸/겨울 옷 칸”으로 나누는 게 도메인형입니다. 여름에 입을 걸 찾을 땐 후자가 훨씬 빠릅니다.

4-3. 의존 방향 — 화살표는 한쪽으로만

```
controller  →  service  →  repository  →  domain
   (얇게)       (규칙)       (DB 접근)      (자기 규칙을 스스로 지킴)
```

domain은 아무에게도 의존하지 않습니다. 계좌 객체는 스프링도, DB도 모르고 "잔액은 마이너스가 되면 안 된다" 같은 자기 규칙만 압니다.

왜 중요한가? 규칙이 여기저기 흩어지면 어디선가 규칙을 안 지키고 잔액을 바꿔버립니다. **잔액 변경은 오직 Account.withdraw() 같은 도메인 메서드로만** 하게 막았습니다.

---

5. 수신 파트 — 맡아주는 돈

5-1. 로그인은 어떻게 되나 (JWT)

문제

서버는 요청이 올 때마다 **"너 누구야?"**를 알아야 합니다. 그런데 매번 아이디/비밀번호를 보내면 위험하고, 서버가 로그인 상태를 전부 기억하면 서버가 여러 대일 때 골치가 아픕니다.

해결: JWT(출입증)

로그인에 성공하면 서버가 **출입증**을 하나 만들어 줍니다. 이 출입증에는 "이 사람은 aaa@bbb.com입니다"라는 내용과, 서버만 아는 비밀로 만든 **서명(도장)**이 찍혀 있습니다.

헤더.내용.서명 ← 점 두 개로 나뉜 긴 글자

다음부터는 요청할 때마다 이 출입증을 같이 보냅니다. 서버는 **도장이 진짜인지만 다시 계산해서** 확인합니다. 진짜면 "우리가 발급한 게 맞네" → 통과.

> **12살 버전**: 놀이공원 손목 밴드예요. 입구에서 한 번 표를 사면 밴드를 채워주고, 이후엔 밴드만 보여주면 됩니다. 직원이 "이 사람 표 샀나?" 명단을 뒤질 필요가 없어요. 밴드에 위조 방지 무늬가 있으니까요.

Refresh Token(연장 티켓)

출입증(Access Token)은 **1시간**만 유효합니다. 짧게 만든 이유는 혹시 도둑맞아도 피해가 적기 때문입니다. 대신 **7일짜리 연장 티켓**을 DB에 저장해 두고, 출입증이 만료되면 이걸로 새로 받습니다.

- 연장 티켓은 **DB에 있어야만 유효** → 서버가 언제든 강제로 없앨 수 있음(로그아웃)
- 재발급할 때마다 **옛 티켓은 버리고 새로 발급**(Rotation) → 훔친 티켓을 오래 못 씀

비밀번호 저장

비밀번호는 **절대 그대로 저장하지 않습니다.** BCrypt라는 방법으로 **되돌릴 수 없게 뭉갠 값**만 저장합니다.

> **12살 버전**: 과일을 갈아서 주스로 만들면 다시 사과로 되돌릴 수 없죠. 로그인할 때는 입력한 비밀번호를 똑같이 갈아서, 저장된 주스와 같은지만 비교합니다.

5-2. 계좌 만들기

계좌번호는 SecureRandom(예측하기 어려운 무작위)으로 만들고, **이미 있는 번호면 다시 뽑습니다**(최대 10번). 10번 다 실패하면 오류를 냅니다.

```
private String generateAccountNumber() {
    for (int i = 0; i < MAX_ACCOUNT_NUMBER_ATTEMPTS; i++) {
        String candidate = randomAccountNumber();
        if (!accountRepository.existsByAccountNumber(candidate)) return candidate;
    }
    throw new BusinessException("계좌번호 생성에 실패했습니다.");
}
```

5-3. 입금·출금·이체

모든 돈 이동은 이 세 가지를 **한 덩어리로** 처리합니다.

1. 잔액을 바꾼다
2. 원장에 기록을 남긴다
3. 둘 다 성공해야 확정한다 (**트랜잭션**)

트랜잭션이 뭔가요? "전부 되거나, 전부 안 되거나" 둘 중 하나만 되게 묶는 것입니다.

12살 버전: 친구에게 사탕을 주면서 내 사탕은 줄고 친구 사탕은 늘어야 하죠. 그런데 내 사탕만 줄고 친구 건 안 늘어나면? 사탕이 증발합니다. 트랜잭션은 **"둘 다 되든가, 둘 다 없던 일로 하든가"**를 보장합니다.

이체 한 번은 원장에 **두 줄**이 남습니다. 보내는 쪽 `TRANSFER_OUT`, 받는 쪽 `TRANSFER_IN`.

5-4. 동시성 — 이 프로젝트의 심장

여기가 가장 중요한 부분입니다. **세 가지 문제를 순서대로 하나씩** 해결했습니다.

문제 1: 잃어버린 갱신 (lost update)

두 사람이 동시에 같은 계좌에서 돈을 뺀다고 해봅시다.

```
잔액 10,000원

A: 잔액 읽음(10,000) ─┐
B: 잔액 읽음(10,000) ─┤  둘 다 10,000을 봤다!
A: 10,000-7,000=3,000 저장
B: 10,000-7,000=3,000 저장   ← A가 뺀 걸 못 봄

결과: 14,000원이 나갔는데 잔액은 3,000원 (7,000원 증발)
```

12살 버전: 두 사람이 동시에 저금통을 보고 "3천 원 있네!"라고 기억한 다음, 각자 3천 원씩 꺼내가는 겁니다. 저금통엔 3천 원밖에 없었는데 6천 원이 나왔어요.

해결: 비관적 락(Pessimistic Lock)

계좌를 읽을 때 **"내가 다 쓸 때까지 아무도 건드리지 마"**라고 자물쇠를 겁니다. SQL로는 `SELECT ... FOR UPDATE`입니다.

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
Optional<Account> findWithLockingById(Long id);   // SELECT ... FOR UPDATE
```

이러면 A가 끝날 때까지 B는 **줄을 서서 기다립니다.** B가 읽을 땐 이미 3,000원으로 바뀐 진짜 값을 봅니다.

실제로 측정한 결과 (잔액 20만 원, 50명이 동시에 1만 원씩 출금 시도 — 정상이면 20명만 성공):

방식	성공 로그	최종 잔액	오차
락 없음	10건	130,000원	**30,000원 증발**
비관적 락	**20건**	**0원**	**0원 (정확)**

락 없는 쪽은 데드락 에러(`SQLState 40001`)도 잔뜩 났습니다.

문제 2: 데드락 (교착 상태)

이체는 **계좌 두 개**를 잠가야 합니다. 그런데 이런 일이 생깁니다.

```
A→B 이체:  A 잠금 → B 잠그려 대기
B→A 이체:  B 잠금 → A 잠그려 대기

서로 상대가 놓기를 영원히 기다림 = 데드락
```

12살 버전: 좁은 문에서 두 사람이 서로 "네가 먼저 비켜"라고 하며 안 움직이는 상황이에요. 영원히 안 끝납니다.

해결: 항상 같은 순서로 잠근다

계좌 ID가 작은 것부터 잠그기로 규칙을 정했습니다. 그러면 두 이체 모두 같은 순서로 잠그니까 서로 기다릴 일이 없습니다.

측정 결과: 반대 방향 동시 이체 **100회 시도, 데드락 0건**.

문제 3: 낡은 값 읽기 (stale read) — 가장 미묘한 버그

이건 진짜 찾기 어렵습니다.

JPA에는 **1차 캐시**가 있습니다. 같은 트랜잭션 안에서 같은 데이터를 두 번 읽으면, 두 번째는 DB에 안 가고 **처음 읽어둔 값을 그냥 줍니다.**

1) 계좌를 그냥 읽음 (잔액 10,000) → 캐시에 저장됨
2) 락을 걸고 다시 읽음 → 캐시가 10,000을 돌려줌 (DB엔 3,000인데!)
3) 낡은 값으로 계산 → 또 틀어짐

> **12살 버전**: 냉장고에 우유가 있는지 확인하고 방에 왔어요. 나중에 다시 "우유 있었나?" 생각할 때, 냉장고를 다시 열지 않고 **기억에 의존**하면? 그 사이 누가 다 마셨을 수도 있잖아요.

해결: 잔액이 걸린 계좌는 첫 조회를 반드시 락 조회로 한다

그런데 문제가 하나 더 있습니다. **받는 계좌의 ID를 알아야 순서를 정하는데, ID를 알려면 읽어야 합니다.** 읽으면 캐시에 들어가고요.

→ **ID만 뽑아오는 조회(프로젝션)**를 씁니다. 엔티티 전체를 안 읽으니 캐시에 들어가지 않습니다.

```
@Query("select a.id from Account a where a.accountNumber = :accountNumber")
Optional<Long> findIdByAccountNumber(String accountNumber);   // ID만!
```

이 세 가지 해결책은 나중에 **대출 상환에서도 똑같이 재사용**됩니다.

---

6. 돈을 다루는 규칙 — BigDecimal 이야기

6-1. 왜 소수가 필요한가

처음엔 잔액을 Long(정수)으로 저장했습니다. 원 단위니까 소수가 필요 없었죠. 그런데 **이자 계산**이 들어오면서 문제가 생깁니다.

1,000만 원 × 연 6% × 30일 / 365일 = 49,315.068...원

소수가 나옵니다. 그리고 컴퓨터에서 소수를 다루는 흔한 타입(double)은 **오차가 있습니다.**

```
0.1 + 0.2 == 0.30000000000000004   // double의 세계
```

> **12살 버전**: 컴퓨터는 1/3을 0.3333...으로 딱 떨어지게 표현 못 해요. 돈 계산에서 이런 오차가 쌓이면 "1원이 어디 갔지?"가 됩니다.

6-2. 해결: BigDecimal + DECIMAL(19,4)

- 자바에서는 **BigDecimal** (오차 없는 소수 계산 타입)
- DB에서는 **DECIMAL(19,4)** (전체 19자리, 소수점 아래 4자리)

6-3. 전환 시점을 왜 그때로 잡았나

이자 계산이 들어오기 "직전"에 전환했습니다.

- 너무 일찍 하면: 필요도 없는데 코드가 복잡해짐
- 너무 늦게 하면: 이미 여러 모듈이 정수를 쓰고 있어서 **다 뜯어고쳐야 함**

6-4. 규칙을 한 곳에 모았다 — MoneyPolicy

반올림 규칙이 여기저기 흩어지면 **같은 계산인데 파일마다 결과가 달라집니다.** 그래서 규칙을 클래스 하나에 못 박았습니다.

```
public final class MoneyPolicy {
    public static final int CALC_SCALE = 10;                // 중간 계산은 소수 10자리
    public static final RoundingMode CALC_ROUND = HALF_UP;   // 중간은 반올림
    public static final int WON_SCALE = 0;                  // 확정 금액은 원 단위
    public static final RoundingMode INTEREST_ROUND = DOWN;  // 이자는 원 미만 절사
    public static final BigDecimal DAYS_PER_YEAR = new BigDecimal("365");
    public static final BigDecimal MAX_LEGAL_RATE = new BigDecimal("0.20"); // 법정최고 20%
}
```

핵심 원칙: 중간 계산은 넉넉하게(10자리), **최종 확정 금액만 원 단위로 자른다.** 중간마다 자르면 오차가 쌓입니다.

---

7. 여신 파트 — 빌려주는 돈

7-1. 대출 상품 (loan_products)

"어떤 조건으로 빌려주는지"를 정의합니다.

컬럼	뜻	예
interest_rate	연이율	0.059 = 연 5.9%
overdue_extra_rate	연체 시 추가 이율	0.03 = +3%
min_amount / max_amount	빌릴 수 있는 범위	100만 ~ 5,000만
max_term_months	최대 기간	60개월
repayment_type	갚는 방식	원리금균등 / 원금균등 / 만기일시

7-2. 상환 방식 3가지 — 그림으로 이해하기

1,000만 원을 연 6%로 12개월 빌린다고 할 때, 갚는 방법이 세 가지입니다.

① 원리금균등 (EqualPayment) — 매달 같은 금액

매달 860,664원씩 12번
 (전부 같은 높이)

내는 돈은 같지만 **속 구성이 변합니다.** 처음엔 이자가 많고 원금이 적다가, 뒤로 갈수록 원금이 많아집니다.

> **12살 버전**: 매달 용돈에서 똑같이 만 원씩 갚는 거예요. 편하죠. 대부분의 주택담보대출이 이 방식입니다.

공식:

$$A = P \times \frac{r(1+r)^n}{(1+r)^n - 1}$$

(P=원금, r=월이율, n=개월수)

② 원금균등 (EqualPrincipal) — 원금만 같게

1회차  (원금 833,333 + 이자 50,000)
2회차  (원금 833,333 + 이자 45,833)
...
12회차  (이자가 점점 줄어듦)

처음엔 많이 내고 갈수록 적게 냅니다. 총 이자는 원리금균등보다 적습니다.

③ 만기일시 (Bullet) — 이자만 내다가 마지막에 몰아서

1~11회차  (이자 50,000원만)
12회차  (원금 1,000만 + 이자)

> **12살 버전**: "일단 매달 조금씩만 내고, 나중에 한 번에 갚을게!" 방식이에요.

7-3. 끝수 조정 — 1원이 남는 문제

여기가 실무에서 진짜 중요한 부분입니다.

이자를 매번 **원 미만 절사**하면, 12번 쌓인 오차 때문에 마지막에 **잔액이 딱 0이 안 됩니다.**

해결: 마지막 회차가 끝수를 흡수한다.

- 12회차 원금 = **남은 잔액 전부**

- 그래서 12회차 상환액은 860,664원이 아니라 **860,662원** (2원 작음)

이걸 빼먹으면 **완납했는데 잔액이 ±수 원 남습니다.** 테스트로 못 박아 뒀습니다.

7-4. 검증을 3층으로 했다

이자 계산은 틀리면 안 되니까 **세 겹으로** 확인했습니다.

1층: 손으로 계산한 값과 비교

수식을 직접 계산해서 "1회차 이자는 50,000원, 원금은 810,664원"을 코드에 그대로 적고 비교합니다.

2층: 240가지 조합에서 불변식 검사

금액 4가지 × 금리 4가지(0% 포함) × 기간 5가지 × 방식 3가지 = **240 조합**에서 다음이 **항상** 성립하는지 확인합니다.

- 회차 원금을 다 더하면 = 대출 원금 (1원도 안 새야 함)
- 마지막 회차 후 잔액 = 정확히 0
- 잔액은 계속 줄어들기만 함 (늘어나면 버그)

> **12살 버전**: 특정 문제 하나만 풀어보는 게 아니라, **"어떤 숫자를 넣어도 이 규칙은 깨지지 않아야 한다"**를 240번 확인하는 겁니다.

3층: 바깥 계산기와 대조

인터넷의 공개 대출이자 계산기와 같은 조건으로 비교했습니다.

월 상환금 860,664원 — 정확히 일치.

회차별 이자는 몇 원 차이가 났는데, 원인을 분석해 보니 그 계산기는 **10원 미만 절사**, FinBank는 **원 미만 절사**라는 정책 차이였습니다. 수식은 같습니다.

7-5. 자동심사 — 룰을 클래스 하나씩

대출 신청이 들어오면 자동으로 심사합니다. 규칙은 2가지입니다.

룰	탈락 조건
`CreditLimitRule`	기존 대출 + 신청액 > 한도(5,000만)
`NoActiveOverdueRule`	이미 연체 중인 대출이 있다

심사와 입력 검증의 경계: 신청할 때 대출금을 받을 계좌를 직접 고르게 되어 있고, 그 계좌의 존재·소유권·잠금 여부는 **심사가 아니라 입력 검증**으로 막습니다(404/403/400). 남의 계좌를 지정한 것은 "심사에서 떨어질 일"이 아니라 애초에 성립하지 않는 요청이기 때문입니다. 심사 룰은 계좌가 유효하다는 전제 위에서 **신용 판단**(한도·연체)만 맡습니다.

> 초기에는 `HasActiveAccountRule`(활성 계좌 보유)이라는 룰이 있었습니다. 계좌를 신청 시 지정하도록 바꾸면서 이 룰은 **절대 발동할 수 없는 코드**가 되었고, 도달 불가능한 코드를 남기는 대신 삭제했습니다.

설계 포인트: 룰을 `if`문 덩어리로 쓰지 않고 **인터페이스 하나 + 구현 클래스 하나씩**으로 만들었습니다.

```java
public interface LoanReviewer {
    Optional<String> reject(ReviewContext context);
    // 탈락이면 룰 코드를 반환, 통과면 비어있음
}
```

스프링이 `List<LoanReviewer>`로 **전부 자동 주입**하므로, **룰을 추가하려면 클래스만 하나 더 만들면 됩니다.** 서비스 코드는 손대지 않습니다.

> **12살 버전**: 심사 항목을 종이 한 장에 다 적어놓으면, 항목 추가할 때마다 그 종이를 고쳐야 해요. 대신 **항목마다 카드 한 장씩** 만들어 상자에 넣으면, 새 항목은 **카드만 추가**하면 됩니다.

7-6. 대출 실행 — 돈이 실제로 나가는 순간

승인된 신청을 **실행**하면 한 트랜잭션 안에서 이 일들이 전부 일어납니다.

1. 신청 상태를 EXECUTED로 바꾼다 (조건부 UPDATE — 뒤에서 설명)
2. 대출 계약(`loan_accounts`) 생성 — **금리·상환방식 스냅샷 저장**
3. 상환 스케줄 **전 회차를 미리 계산해서 저장** (12개월이면 12줄)
4. 연결된 입출금 계좌에 **원금 입금**
5. 대출 원장에 EXECUTION 기록

왜 스케줄을 미리 다 저장하나? "몇 회차에 얼마 내야 하는지"가 계약 시점에 확정되어야 하고, 나중에 계산식이 바뀌어도 **이미 맺은 계약의 스케줄은 변하면 안 되기 때문**입니다.

7-7. 상환 — 갚기

- **회차 상환**: 이번 달 낼 돈을 계좌에서 빼고, 회차를 PAID로, 대출 잔액을 원금만큼 줄임
- **순서 규칙**: 1회차가 남아있는데 2회차를 내려 하면 거부 (409)
- **전액 중도상환**: 남은 원금 + 경과이자 + 중도상환수수료를 한 번에

중도상환수수료 계산

수수료 = 잔여원금 × 수수료율 × (잔존일수 ÷ 총 대출일수)

남은 기간이 짧을수록 수수료가 줄어듭니다(체감 방식). 만기일에 갚으면 수수료 0원입니다.

> **12살 버전**: 은행은 "10달 동안 이자 받을 계획"으로 돈을 빌려줬는데, 5달 만에 갚으면 계획이 틀어져요. 그 손해를 조금 물어주는 게 수수료입니다. 다만 만기가 다 되어 갚으면 손해가 없으니 안 받습니다.

남은 회차는 지우지 않고 `CANCELED`로 남깁니다. "원래 어떤 계획이었는지"도 사실이기 때문입니다(원장 철학의 연장).

---

8. 멱등키 — “두 번 눌러도 한 번만”

8-1. 문제 상황

사용자가 **상환 버튼을 두 번 눌렀습니다.** 또는 인터넷이 끊겨서 앱이 자동으로 재시도했습니다. 서버에는 **똑같은 요청이 두 번** 도착합니다.

돈이 두 번 빠지면 안 됩니다.

> **12살 버전**: 자판기에서 버튼을 두 번 눌렀는데 음료수가 두 개 나오고 돈도 두 번 빠지면 억울하잖아요. “이미 처리한 요청이면 그냥 아까 그 음료수를 다시 보여줘”가 멱등입니다.

8-2. 해결: 요청마다 고유 번호표

클라이언트(앱/웹)가 **요청 하나마다 UUID(고유 번호)**를 만들어 헤더에 담아 보냅니다.

```
Idempotency-Key: 620f6171-0ece-4307-a953-77d732d3ca3d
```

서버는 이 번호표를 보고 판단합니다.

- **처음 보는 번호** → 처리하고, 결과를 저장해 둠
- **이미 처리한 번호** → 저장해 둔 결과를 **그대로 다시 돌려줌** (돈은 안 움직임)
- **처리 중인 번호** → 409 (잠시 후 다시)
- **같은 번호인데 내용이 다름** → 422 (번호표 재사용은 거부)

8-3. 화면에서 키를 어떻게 다뤄야 하나

여기서 **처음에 실수**를 했습니다. API를 호출할 때마다 새 UUID를 만들고 있었습니다.

```
// 잘못된 방식: 호출마다 새 키 → 멱등이 전혀 안 걸림
fetch(url, { headers: { 'Idempotency-Key': crypto.randomUUID() } })
```

올바른 방식은 “사용자 행동 1회 = 키 1개”입니다. 버튼을 누른 순간 키를 만들고, 네트워크 실패로 재시도할 때는 **같은 키를 그대로 다시** 보냅니다.

8-4. 트랜잭션 경계 — 가장 어려운 부분

멱등키를 제대로 만들려면 **“어디까지가 한 덩어리인가”**를 정교하게 나눠야 합니다.

단계	전파 속성	왜
키 선점(INSERT)	`REQUIRES_NEW`(새 트랜잭션)	본 처리와 같이 묶으면 롤백 시 **키도 사라져** 동시 재시도를 못 막는다
결과 확정	`MANDATORY`(같은 트랜잭션)	“돈이 움직인 사실”과 “저장된 응답”이 **같은 커밋에 묶여야** 한다
실패 시 해제	`REQUIRES_NEW`	롤백되는 트랜잭션에 얹으면 **삭제도 함께 취소**된다

> **12살 버전**: 도서관에서 책을 빌릴 때 “예약 도장”을 먼저 찍어야 남이 못 가져가요. 그런데 그 도장을 **대출 절차와 한 묶음**으로 만들면, 대출이 취소될 때 도장도 지워져서 그 틈에 남이 채갑니다. 그래서 **도장은 따로** 찍습니다.

8-5. 경쟁의 심판은 DB에게 맡긴다

“먼저 조회해서 없으면 저장”은 **동시에 오면 둘 다 통과합니다.**

```
A: 키 있나? → 없음
B: 키 있나? → 없음        (A가 아직 저장 전)
A: 저장 → 성공
B: 저장 → 성공            ← 둘 다 통과! 실패
```

그래서 **UNIQUE 제약**을 걸어두고 **그냥 INSERT를 시도**합니다. DB가 **반드시 하나만** 통과시킵니다.

> **12살 버전**: 의자 뺏기 게임에서 "빈자리 있나 두리번거리고 앉기"는 두 명이 같이 앉을 수 있어요. 그냥 **앉아버리면** 의자는 하나니까 한 명만 앉습니다. 심판을 사람이 아니라 **의자에게 맡기는** 거죠.

8-6. 실제 측정 결과

100개 스레드가 같은 회차를 동시에 상환 시도 (1회차 총액 860,205원):

시나리오	성공 응답	409 충돌	기타 예외	원장	실제 출금	잔액 감소
서로 다른 100개 요청	**1**	99	0	1건	1회	860,205원
같은 키 100번 재시도	70	30	0	**1건**	**1회**	860,205원

두 번째 줄이 멱등키의 존재 이유입니다. 100개 중 70개가 **성공 응답을 받았지만 돈은 한 번만 움직였습니다.** 69개는 저장된 응답의 재생이고, 30개는 선점 경쟁 중이라 409를 받았습니다.

8-7. 이중 처리 방어 — 층마다 다른 걸 막는다

사고	1차 방어	최종 방어
같은 신청 이중 실행	조건부 UPDATE (0행이면 409)	`application_id` UNIQUE
같은 회차 이중 상환	조건부 UPDATE	`idempotency_key` UNIQUE
버튼 두 번 / 재시도	멱등키 선점	위 두 제약

조건부 UPDATE가 왜 핵심인가

MySQL 기본 격리수준(REPEATABLE READ)에서 **평범한 SELECT는 트랜잭션 시작 시점의 사진(스냅샷)을 봅니다.** 락을 기다렸다 통과한 두 번째 요청이 **"이미 상환된 회차"를 아직 안 낸 것으로 볼 수 있다**는 뜻입니다.

반면 **UPDATE는 항상 최신 값을 기준으로** 조건을 평가합니다.

```
UPDATE repayment_schedules
   SET status = 'PAID'
 WHERE id = ? AND status IN ('PENDING','OVERDUE')   -- 조건을 UPDATE에 넣는다
```

바뀐 행 수가 **0이면 "이미 누가 했다"** → 409로 거부.

> **핵심**: **상태 확인과 상태 변경을 DB의 한 문장으로 합치는 것**이 경쟁을 없애는 방법입니다. 확인과 변경 사이에 틈이 있으면 그 틈으로 사고가 들어옵니다.

8-8. 대출 상환의 락 순서

상환은 **계좌와 대출 두 자원**을 동시에 잠급니다. 전역 규칙은 **Account → LoanAccount 순서**입니다.

그런데 연결계좌 ID는 대출을 읽어야 압니다 → 수신에서 배운 대로 **ID만 프로젝션으로** 먼저 읽습니다. 엔티티를 먼저 읽으면 1차 캐시 때문에 락 조회가 낡은 값을 돌려주니까요.

---

9. 검색과 인덱스 — 느린 걸 빠르게

9-1. 문제: 조건이 7개인 검색

대출 목록을 상태·상환방식·상품·실행일 구간·원금 구간으로 자유롭게 조합해서 찾고 싶습니다. 조건을 넣을 수도, 안 넣을 수도 있습니다.

조합의 수가 100가지를 넘습니다. 이걸 if문으로 쓰면 코드가 지옥이 됩니다.

9-2. 해결: QueryDSL

조건 하나 = 메서드 하나로 만들고, 값이 없으면 null을 반환합니다. QueryDSL의 where()는 **null을 만나면 그 조건을 그냥 건너뜁니다.**

```
private BooleanExpression statusEq(LoanAccountStatus status) {
    return status == null ? null : loanAccount.status.eq(status);
}
```

> **12살 버전**: 검색 조건을 카드로 만들어서, 빈 카드는 그냥 버리는 거예요. 조합이 몇 가지든 카드는 조건 개수만큼만 있으면 됩니다.

9-3. 인덱스 — 책의 목차

인덱스가 없으면 DB는 표를 **처음부터 끝까지 다 훑습니다**(풀스캔).

> **12살 버전**: 두꺼운 책에서 "고양이"라는 단어를 찾을 때, 1쪽부터 넘기면 오래 걸리죠. 뒤에 **색인(찾아보기)**이 있으면 "고양이 → 45쪽"으로 바로 갑니다. 인덱스가 바로 그 색인입니다.

9-4. 실제로 재봤다 (10만 건)

조건: 한 회원이 대출 10만 건 보유 / "최근 30일 안에 실행된 정상 대출"(전체의 약 2.7%)

쿼리	인덱스	실행 방식	조사한 행 수	평균 시간
COUNT	없음	풀스캔(ALL)	96,880	26.62ms
COUNT	**복합 인덱스**	범위 스캔(range)	**2,664**	**1.04ms**

조사 행 수 36배 감소, 시간 26배 단축.

9-5. 인덱스 컬럼 순서가 중요하다

인덱스는 (member_id, status, executed_at) 순서로 만들었습니다. 순서에 이유가 있습니다.

> **핵심 규칙**: **등치 조건(=)을 앞에, 범위 조건(>, <, BETWEEN)을 뒤에.**
>
> **왜?** 범위 조건 뒤에 오는 컬럼은 **탐색 범위를 더 좁히지 못하기 때문**입니다.
>
> **12살 버전**: 반 친구를 찾을 때 "3학년 → 2반 → 키 순서"로 정렬돼 있으면 빨리 찾죠. 그런데 "키 순서 → 3학년 → 2반"으로 정렬하면? 키로 한 번 범위를 잡은 다음엔 학년·반이 뒤죽박죽이라 소용이 없습니다.

9-6. 정직하게 기록한 한계

정렬이 붙은 목록 쿼리(ORDER BY id DESC LIMIT 10)는 **인덱스가 있으나 없으나 PK를 탔습니다.**

옵티마이저가 "PK 역순으로 훑다가 10건 채우면 끝"이 더 빠르다고 판단한 겁니다. 지금은 조건 일치율이 2.7%라 빠르지만, **데이터 분포에 의존하는 계획**입니다.

→ **커버링 인덱스나 커서 페이징이 필요한 지점**으로 문서에 남겨뒀습니다. 이런 한계를 숨기지 않고 적는 게 중요합니다.

---

10. 배치 — 밤에 혼자 도는 프로그램

10-1. 배치가 뭔가요?

사람이 시키지 않아도 **정해진 시간에 자동으로 도는 프로그램**입니다. FinBank는 **매일 새벽 2시**에 두 개가 순서대로 돕니다.

1. **연체 감지** — 납기일이 지났는데 안 낸 회차를 찾아 OVERDUE로 표시 + 연체 이력 생성
2. **연체이자 가산** — 진행 중인 연체에 하루치 이자를 더함

순서가 중요합니다. 오늘 새로 연체된 건은 감지가 이력을 만들어줘야 가산 대상이 되기 때문입니다.

10-2. 가장 조심해야 할 함정: 페이징 밀림

이 배치들은 **읽은 행의 상태를 바꿔서, 자기 조회 조건에서 스스로 빠져나갑니다.** 여기서 흔한 방식(offset 페이징)을 쓰면 **조용히 데이터가 빠집니다.**

```
1페이지(0~99번) 처리 → 이 100건이 조건에서 빠짐
2페이지(offset 100) 조회
   → 원래 100~199번이던 행들이 0~99번으로 당겨져 있음
   → 그 100건을 통째로 건너뛴다!
```

10만 건 중 절반만 처리하고도 배치는 "정상 종료"로 끝납니다.

> **12살 버전**: 사탕 100개를 한 줄로 세워놓고 "앞에서 10개씩 가져가기"를 하는데, 가져간 사탕은 줄에서 빠져요. 그럼 "11번째부터"라고 세면 이미 줄이 당겨져서 **엉뚱한 사탕부터** 가져갑니다. 중간에 건너뛴 사탕이 생기죠.

해결: 커서 방식 — "마지막으로 읽은 id **다음부터**" 읽습니다.

```
// 위치(offset)가 아니라 마지막 id를 기억한다
findOverdueCandidates(baseDate, lastId, pageable)
```

이러면 앞이 빠져도 상관없고, offset이 커질수록 느려지는 문제도 없어집니다. **청크 경계를 넘는 250건 테스트**로 못 박아 뒀습니다.

10-3. 멱등을 세 겹으로

배치가 실수로 두 번 돌아도 사고가 나면 안 됩니다.

층	장치	막는 것
프레임워크	Spring Batch JobInstance	같은 파라미터(기준일)의 재실행 거부
감지 배치	`status = PENDING` 조회 조건	이미 전이된 회차는 조회조차 안 됨
가산 배치	`lastAccruedAt < 기준일` 조건	오늘 이미 가산한 이력은 제외

원리는 하나입니다: 처리 완료 표시를 데이터에 남기고, 그 표시를 읽기 조건에 넣는다.

> **12살 버전**: 숙제를 다 하면 공책에 도장을 찍어요. 다음에 "도장 없는 것만" 찾으면, 실수로 두 번 확인해도 이미 한 건 다시 안 합니다.

10-4. 연체이자 기산점 — 구간이 이어지게

`lastAccruedAt`(마지막 가산일)이 있으면 거기서부터, 없으면 납기일부터 계산합니다.

```
7/1 연체 시작
7/11 배치 → 10일치 가산, lastAccruedAt = 7/11
7/16 배치 → 7/11~7/16의 5일치만 가산  ← 겹치지도 비지도 않음
```

배치가 사흘 밀렸다면 다음 실행에서 3일치가 한 번에 붙습니다.

10-5. 청크 사이즈 — 재보고 정했다

청크(chunk)란 "한 트랜잭션에 몇 건을 묶을 것인가"입니다. 크게 잡으면 커밋(저장 확정) 횟수가 줄어 빨라지지만, 실패했을 때 되돌아가는 범위가 커집니다.

5만 건 연체 감지 실측:

청크 크기	처리시간	커밋 횟수	건당
100	37,969ms	500	759µs
500	12,418ms	100	248µs
1000	**9,927ms**	**50**	**198µs**

100 → 1000으로 3.8배 빨라졌습니다. 그런데 자세히 보면:

- 100 → 500: **3.1배** 빨라짐
- 500 → 1000: **1.25배**밖에 안 빨라짐

수확이 급격히 체감합니다. 커밋 비용이 지배적인 구간을 벗어나면, 그다음은 쿼리와 메모리 비용이라 청크를 키워도 얻는 게 적습니다.

그래서 정답이 아니라 트레이드오프입니다. 기본값 100을 유지하되 잡 파라미터로 조정할 수 있게 열어뒀습니다.

10-6. 스케줄러의 한계를 정직하게 적었다

`@Scheduled`는 **인스턴스마다 독립적으로 돕니다.** 서버를 2대로 늘리면 같은 배치가 동시에 두 번 트리거됩니다.

- 지금 안전한 이유: **단일 인스턴스 배포**이고, 위의 멱등 3겹이 있어서 중복 실행이 사고로 이어지지 않음
- 다중화한다면: 분산 락(ShedLock)이나 외부 스케줄러(쿠버네티스 CronJob)로 실행 주체를 하나로 만들어야 함
- **지금 도입하지 않는 이유**: 다중 인스턴스를 증명할 환경이 없어서

---

11. 예외 처리 — 오류를 어떻게 알려주나

11-1. 상태 코드로 구분한다

코드	뜻	예
400	요청이 잘못됨	잔액 부족, 금액이 0 이하
403	권한 없음	남의 계좌를 보려 함
404	없음	없는 계좌
409	**충돌**	이미 실행됨, 이미 상환됨, 처리 중
422	**처리 불가**	멱등키 재사용, 금액이 상품 범위 밖

409와 422를 따로 만든 이유: 400 하나로 뭉뚱그리면 화면에서 "다시 시도해도 되는 오류"와 "다시 시도하면 안 되는 오류"를 구분할 수 없습니다.

11-2. ErrorCode로 한 곳에 모았다

```
public enum ErrorCode {
    ALREADY_EXECUTED(CONFLICT, "이미 실행된 신청입니다"),
    ALREADY_REPAID(CONFLICT, "이미 상환된 회차입니다"),
    OUT_OF_ORDER_INSTALLMENT(CONFLICT, "이전 회차를 먼저 상환해야 합니다"),
    IDEMPOTENT_REQUEST_IN_PROGRESS(CONFLICT, "처리 중인 요청입니다"),
    IDEMPOTENCY_KEY_MISMATCH(UNPROCESSABLE_ENTITY, "같은 키로 다른 요청을 보냈습니다"),
    ...
}
```

서비스는 **의미에 맞는 예외만 던지고**, 상태 코드 매핑은 `GlobalExceptionHandler` 한 곳에 모았습니다.

11-3. 점진 이관 (expand-contract)

기존에는 `BusinessException(String)`으로 문자열만 던지고 있었습니다. 이걸 전부 한 번에 바꾸면 위험하니까:

1. 기존 생성자를 **그대로 두고**
2. `ErrorCode`를 받는 생성자를 **추가**

→ 기존 코드는 하나도 안 고치고 새 코드부터 새 방식을 씁니다.

> **12살 버전**: 좁은 다리를 새 다리로 바꿀 때, 헌 다리를 부수고 새로 짓는 게 아니라 **옆에 새 다리를 놓고 사람들을 천천히 옮기는** 방식입니다.

---

12. 테스트 — 333개가 무엇을 지키나

12-1. 철학: 개수보다 "위험한 곳"

모든 코드를 다 테스트하지 않았습니다. **틀렸을 때 돈이 사라지는 곳**에 집중했습니다.

12-2. 테스트 파일별 역할 (전체)

수신 — 동시성과 규칙

파일	개수	무엇을 지키나
ConcurrencyBenchmarkTest	1	**락 없음 vs 비관적 락** 비교표 출력 (증발 금액을 숫자로)
WithdrawConcurrencyTest	1	10만 원 계좌에 7만 원 출금 2번 동시 → **1번만 성공**
AccountConcurrencyTest	1	동시 이체 → 1건만 성공
AccountServiceDeadlockTest	1	반대 방향 동시 이체 100회 → **데드락 0건**
TransactionLogTest	3	입금·출금·이체 시 원장이 정확히 남는가 (이체는 2줄)
TransferRuleTest	6	잔액부족·같은계좌·남의계좌·잠금계좌 거부

예외

파일	개수	무엇을 지키나
GlobalExceptionHandlerTest	3	예외 → 상태코드 매핑, 레거시 호환

여신 — 계산 엔진 (DB 없이 순수 자바)

파일	개수	무엇을 지키나
ScheduleGeneratorTest	12	3가지 상환방식의 회차별 원금·이자·잔액 **확정값** + 끝수 조정
ScheduleGeneratorPropertyTest	**240 조합**	"원금 합계 = 대출원금, 최종잔액 = 0, 잔액 단조감소"가 **항상** 성립
AccruedInterestCalculatorTest	5	경과이자 계산 + 원 미만 절사
OverdueAndPrepaymentCalculatorTest	8	연체이자(법정최고 20% 캡) + 중도상환수수료(체감)
LoanApplicationTest	6	신청 상태 전이 규칙 (APPLIED→APPROVED→EXECUTED)

여신 — 통합 (실제 DB)

파일	개수	무엇을 지키나
LoanApplicationServiceTest	8	자동심사 3룰 + 한도·기간 검증 + 취소
LoanExecutionServiceTest	7	실행 정합성, **멱등 재생**, 이중 실행 409, 키 재사용 422, 실패 시 키 해제
LoanRepaymentServiceTest	9	회차 상환, 순서 규칙, 멱등, 완납, 전액 중도상환
LoanRepaymentConcurrencyTest	2	**100스레드 동시 상환 → 1건만**, 같은 키 100회 → 돈은 1번만
LoanSearchTest	8	조건 조합, 경계 포함, 타인 대출 미노출, 페이징

배치

파일	개수	무엇을 지키나
`OverdueDetectionJobTest`	5	감지 정확성, 멱등, **청크 경계 250건 누락 없음**, 기준일 파라미터
`OverdueInterestAccrualJobTest`	7	계산기와 금액 일치, 멱등, **구간 연속성**, 밀린 배치 따라잡기, **상환 시 가산 중단**

성능 측정 (perf 태그 — 평소엔 제외)

파일	무엇을 재나
`LoanSearchPerformanceTest`	10만 건 인덱스 전/후 EXPLAIN + 응답시간
`BatchChunkSizePerformanceTest`	5만 건 청크 100/500/1000 처리시간

12-3. 왜 진짜 MySQL로 테스트하나

메모리 DB(H2)를 쓰면 테스트가 훨씬 빠릅니다. 그런데 **안 씁니다.**

이 프로젝트가 증명하려는 것들이 대부분 DB 엔진의 특성이기 때문입니다.

- 비관적 락(`SELECT ... FOR UPDATE`)의 동작
- 조건부 UPDATE가 최신 값을 보는 것
- REPEATABLE READ 스냅샷

H2로 바꾸면 **검증 자체가 무의미해집니다.**

12-4. 성능 테스트를 분리한 이유

10만 건을 적재하는 측정은 1분 넘게 걸립니다. 매 커밋마다 돌리면 **개발 피드백이 느려집니다.**

```
test    { useJUnitPlatform { excludeTags 'perf' } }   // 평소: perf 제외
perfTest { useJUnitPlatform { includeTags 'perf' } }   // 필요할 때만
```

CI 시간 = 개발 피드백 속도입니다.

12-5. 테스트 DB 분리

테스트는 `DatabaseCleaner`로 **모든 표를 비웁니다.** 개발용 DB를 그대로 쓰면 `./gradlew test` 한 번에 개발하며 만든 데이터가 전부 사라집니다.

→ 테스트는 **`finbank_test`** 스키마, 개발 서버는 **`finbank`** 스키마로 갈랐습니다.

왜 중요한가? 테스트가 "돌리기 무서운 것"이 되면 **결국 안 돌리게 됩니다.**

---

13. 자동화와 실행 환경

13-1. CI (GitHub Actions) — 자동 감독관

코드를 올릴 때마다 **자동으로 시험을 봅니다.**

```
push(main/develop) 또는 PR
  → 백엔드: MySQL 컨테이너 띄우고 333개 테스트
  → 프론트엔드: 빌드
  → 하나라도 빨간불이면 머지 못 함
```

왜 필요한가? 테스트가 333개까지 커지면 수동 실행에 의존할 때 언젠가 깨진 채로 쌓입니다.

13-2. Docker Compose — 한 번에 켜기

```
git clone ...
cd FinBank
docker compose up      # 백엔드 :8080, MySQL :3306
```

MySQL 버전, 문자셋, 초기 스키마 두 개까지 **코드로 고정**했습니다. 리뷰어가 5분 안에 실행해볼 수 있습니다.

healthcheck를 걸어서 **DB가 준비된 뒤에** 앱이 뜨게 했습니다. 없으면 기동 순서 때문에 간헐적으로 실패합니다.

13-3. 설정 우선순위

```
시스템 프로퍼티 / 환경변수    ← 가장 강함
  ↓
application-{profile}.yml
  ↓
application.yml              ← 가장 약함
```

- 로컬: gitignore된 `application-local.yml`이 계정 정보를 채움
- CI/Docker: **환경변수**가 채움
- **비밀은 절대 git에 올리지 않습니다.**

---

14. 실패 기록 8건 — 무엇이 왜 깨졌나

이 장이 이 문서에서 가장 값진 부분일지도 모릅니다. **성공한 코드보다 실패한 과정이 실력을 보여주기 때문입니다.**

① BigDecimal 전환 후 테스트 전멸

- **문제**: 금액 타입을 바꿨더니 멀쩡하던 테스트가 무더기로 실패
- **원인**: DB `DECIMAL(19,4)`는 `1000.0000`으로 조회되는데, `equals`는 **스케일까지 비교**한다. `1000 != 1000.0000`
- **해결**: 비교를 전부 `compareTo` 계열로 (테스트는 `isEqualByComparingTo`)
- **교훈**: 값이 같다는 것과 표현이 같다는 것은 다르다

② Flyway 유령 마이그레이션

- **문제**: 체크섬 불일치 + 새로 만든 V3가 실행 안 됨
- **원인**: 로컬 DB에 **과거 실험 브랜치의 V3·V4 기록**이 남아 있었음
- **해결**: 로컬 한정으로 clean 후 재마이그레이션
- **교훈**: **적용된 마이그레이션 파일은 절대 수정 금지.** 공유 DB라면 되돌리지 말고 새 버전으로 앞으로 고친다(forward-fix)

③ 코드를 안 고쳤는데 어느 날 깨지는 테스트

- **문제**: `Data too long for column 'account_number'`로 테스트 10개 실패
- **원인**: 테스트가 계좌번호를 `"prefix-" + System.nanoTime()`으로 만들고 있었는데, **nanoTime은 컴퓨터를 켜둔 시간이 길수록 자릿수가 늘어난다.** 어느 날 `VARCHAR(20)`을 넘어섬
- **해결**: 12자리 고정 생성기(`TestAccountNumbers`)로 교체
- **교훈**: 테스트 데이터도 **길이가 보장되는 방식**으로 만들어야 한다

④ 인덱스를 지울 수 없다 (MySQL 에러 1553)

- **문제**: 성능 측정을 위해 인덱스를 지우려는데 "외래키가 이 인덱스를 쓰고 있다"며 거부
- **원인**: 복합 인덱스를 만들자 MySQL이 **FK용으로 자동 생성했던 인덱스를 조용히 제거**했고, 그래서 복합 인덱스가 FK를 지탱하는 유일한 인덱스가 됨
- **해결**: DROP/CREATE 대신 **`IGNORE INDEX` 힌트**로 "인덱스를 안 쓴 척"하며 측정
- **뜻밖의 이득**: 진짜 풀스캔(type=ALL)이 나와서 비교가 더 정확해짐

⑤ 배치 테스트가 두 번째부터 실패

- **문제**: `JobInstanceAlreadyCompleteException`
- **원인**: 잡 파라미터가 같으면 Spring Batch가 **재실행을 거부**한다. 테스트마다 같은 값을 썼음
- **해결**: 테스트에는 `UUID`를 파라미터에 추가
- **판단**: 이건 버그가 아니라 **프레임워크의 중복 실행 방지 기능**이므로, 스케줄러에는 그대로 두었다

⑥ 배치를 만들자 드러난 상환의 빈틈

- **문제**: 연체 회차를 상환해도 **연체이자가 계속 붙는 구조**였다
- **원인**: 상환 로직이 회차와 대출 상태만 되돌리고 **`OverdueHistory`를 닫지 않았다.** C3 시점에는 연체이력을 만드는 주체(배치)가 없어서 **증상이 없었다** — 코드는 틀렸지만 드러나지 않았던 것
- **해결**: 상환 시 이력을 `resolve`하고, "상환 → 가산 중단"을 통합 테스트로 고정
- **교훈**: **기능 A와 B가 같은 데이터를 반대 방향으로 다룰 때, B가 없는 동안 A의 버그는 잠복한다**

⑦ PAT 권한 부족으로 push 거부

- **문제**: `.github/workflows/ci.yml`을 올리려는데 "workflow 권한이 없다"며 거부

- **해결**: 기존 토큰에 `workflow` 체크박스만 추가 (토큰 문자열이 그대로라 키체인 재입력 불필요)

⑧ 테스트 설정 파일 하나 추가했더니 컨텍스트 전체가 죽었다

- **문제**: 테스트 DB를 분리하려고 `src/test/resources/application.yml`에 URL만 적었는데, 통합 테스트 **53개가 전부** `Access denied for user 'bagjeyeong'`로 실패. 적은 적도 없는 계정 이름이었다
- **원인**: main과 test 모두 `classpath:/application.yml`이다. **같은 이름·같은 위치라 Spring은 둘을 합치지 않고 테스트 것 하나만 읽는다.** URL 한 줄만 남고 username·password·JPA·Flyway 설정이 **통째로 사라져** 접속 계정이 비어 OS 사용자명으로 시도한 것
- **해결**: 설정 파일을 없애고 **Gradle 테스트 태스크의 시스템 프로퍼티**로 URL만 주입. 시스템 프로퍼티는 모든 설정 파일보다 우선하면서 나머지는 그대로 살려둔다
- **교훈**: **덮어쓰려면 같은 파일을 다시 만들지 말고, 우선순위가 더 높은 곳에서 그 키만 지정한다.** 에러 메시지에 "내가 적은 적 없는 값"이 나오면 대개 설정이 사라졌다는 신호다

---

15. 면접 예상 질문과 답

Q1. 이 프로젝트에서 가장 어려웠던 문제는?

stale read(낡은 값 읽기)입니다. 락을 걸었는데도 값이 틀어져서 오래 헤맸습니다. 원인은 JPA 1차 캐시였습니다. 락을 걸기 전에 엔티티를 한 번 읽으면 캐시에 남아, 락 조회가 DB에 가지 않고 낡은 값을 돌려줍니다.

해결은 **"잔액이 걸린 계좌는 첫 조회를 반드시 락 조회로"**입니다. 그런데 순서를 정하려면 상대 계좌 ID가 필요해서, **ID만 뽑는 프로젝션 조회**를 따로 만들었습니다. 이 패턴은 나중에 대출 상환에서도 그대로 재사용했습니다.

Q2. 멱등키를 왜 직접 만들었나요?

돈이 움직이는 API는 **재시도가 필연**이기 때문입니다. 네트워크 타임아웃이 나면 클라이언트는 성공인지 실패인지 알 수 없고, 재시도할 수밖에 없습니다.

핵심은 **트랜잭션 경계**였습니다. 키 선점은 본 처리와 **다른 트랜잭션**이어야 합니다. 같이 묶으면 롤백될 때 키도 사라져서 동시 재시도를 못 막습니다. 반대로 결과 저장은 **같은 트랜잭션**이어야 합니다. "돈이 움직인 사실"과 "저장된 응답"이 갈라지면 안 되니까요.

Q3. 왜 낙관적 락이 아니라 비관적 락인가요?

충돌이 잦고, 충돌 시 재시도 비용이 큰 도메인이기 때문입니다. 같은 계좌에 동시 접근이 몰리면 낙관적 락은 대부분 실패하고 재시도가 폭증합니다. 반면 계좌 하나에 걸리는 락은 **매우 짧게 유지**되므로 줄을 세우는 편이 낫습니다.

실제로 벤치마크에서 **비관적 락이 오히려 더 빨랐습니다**(락 없는 쪽은 데드락 → 롤백 → 재시도로 지연).

Q4. 조건부 UPDATE를 쓴 이유는?

MySQL REPEATABLE READ에서 **일반 SELECT는 스냅샷을 읽기 때문**입니다. 락을 기다렸다 통과한 두 번째 요청이 "이미 상환된 회차"를 아직 안 낸 것으로 볼 수 있습니다.

UPDATE는 항상 최신 커밋본을 기준으로 조건을 평가하므로, **상태 확인과 변경을 한 문장으로 합치면** check-then-act 경쟁 자체가 사라집니다.

Q5. 배치에서 가장 신경 쓴 부분은?

Reader 설계입니다. 이 배치들은 읽은 행의 상태를 바꿔 자기 조회 조건에서 스스로 빠져나갑니다. offset 페이징을 쓰면 앞이 빠진 만큼 뒤가 당겨져 **통째로 건너뛰는데, 그러고도 배치는 정상 종료합니다.** 조용해서 더 위험한 버그입니다.

커서 방식(마지막 id 다음부터)으로 해결하고, **청크 경계를 넘는 250건 테스트**로 고정했습니다.

Q6. 왜 Redis를 안 썼나요?

필요를 증명할 수 없어서입니다. 분산 락이 필요한 상황은 다중 인스턴스인데, 이 프로젝트는 단일 인스턴스 배포입니다. 대신 **멱등 3겹**으로 중복 실행이 사고로 이어지지 않게 만들고, "다중화한다면 ShedLock이나 외부 스케줄러가 필요하다"를 문서에 적어뒀습니다.

기술을 넣는 것보다 **왜 안 넣었는지 설명할 수 있는 것**이 더 중요하다고 생각합니다.

Q7. 성능을 어떻게 측정했나요?

추측하지 않고 EXPLAIN과 시간을 직접 쟀습니다.

- 인덱스: 10만 건에서 조사 행 96,880 → 2,664 (36배), 26.6ms → 1.0ms
- 청크: 5만 건에서 37,969ms → 9,927ms (3.8배)

그리고 **한계도 같이 기록했습니다.** 정렬 목록 쿼리는 인덱스를 안 타고 PK를 탔는데, 이건 데이터 분포에 의존하는 계획이라 커버링 인덱스나 커서 페이징이 필요한 지점이라고 적어뒀습니다.

Q8. 다음에 개선한다면?

세 가지를 조건과 함께 남겨뒀습니다.

1. **커버링 인덱스 / 커서 페이징** — 데이터 분포가 나빠지면 정렬 목록 쿼리가 먼저 무너집니다
2. **분산 락 또는 외부 스케줄러** — 배치를 다중 인스턴스에서 돌릴 때
3. **SSE 전환** — 이체 수신 알림의 20초 폴링을 푸시로

---

16. 한 장 요약

이 프로젝트가 증명하는 것

주제	증명 방법	숫자
동시성 제어	락 없음 vs 비관적 락 벤치마크	3만 원 증발 vs **0원 오차**
데드락 방지	반대 방향 동시 이체 100회	**데드락 0건**
멱등성	같은 키 100회 동시 재시도	70개 성공 응답, **원장 1건**
이자 정확성	240조합 불변식 + 외부 계산기 대조	월 상환금 **완전 일치**
인덱스 효과	10만 건 EXPLAIN 전/후	**36배 / 26배**
배치 처리량	5만 건 청크 3종 비교	**3.8배** (수확 체감 확인)
자동 검증	CI에서 매 커밋	**테스트 333개**

전 구간 흐름

```
[회원가입] → [로그인/JWT] → [계좌 개설] → [입금]
                                              ↓
                          [대출 상품 조회] → [신청] → [자동심사 3룰]
                                                        ↓ 승인
                                          [실행: 스케줄 생성 + 계좌 입금]
                                                        ↓
                                [회차 상환] ←→ [전액 중도상환] ←→ [완납]
                                      ↓ 안 갚으면
                          [매일 새벽 2시 배치: 연체 감지 → 연체이자 가산]
                                      ↓ 갚으면
                                [연체 이력 해소 → 가산 중단]
```

처음부터 다시 만든다면 — 순서

```
1. 목표 한 문장 고정    "잔액이 절대 틀어지면 안 된다"
2. ERD 먼저             데이터가 모든 로직의 토대
3. 인증                 모든 API가 "로그인한 사람" 기준
4. 기본 거래            락 없이 happy path부터
5. 동시성               문제 재현 → 락 → 데드락 → stale read (한 겹씩)
6. 금액 타입 전환       소수 연산이 필요해지기 직전에
7. 여신                 상품 → 신청/심사 → 계산엔진 → 실행/상환
8. 성능                 필요가 생긴 뒤에 재고 고친다
9. 배치                 도메인이 요구할 때 도입
10. 마감                CI · Docker · 문서
```

각 단계는 앞 단계가 있어야 성립합니다. 그리고 매 단계 끝에 **숫자 하나 + 트러블슈팅 하나 + 문서 갱신**을 남겼습니다.

FinBank — 2026.07.28 기준. 테스트 333개, 마이그레이션 V1~V6, 전 구간 CI 검증.

부록 A. 코드 상세 레퍼런스

여기서는 **실제 코드를 계층별로** 하나씩 봅니다. 각 코드 아래에 "이게 왜 이렇게 생겼는지"를 붙였습니다.

A-1. 도메인 — 자기 규칙을 스스로 지키는 객체

Account (계좌)

```
@Entity @Table(name = "accounts")
public class Account {
    @Id @GeneratedValue(strategy = IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 20)
    private String accountNumber;          // 계좌번호 (유일)

    @ManyToOne(fetch = LAZY) @JoinColumn(name = "member_id")
    private Member member;                 // 주인

    @Column(nullable = false, precision = 19, scale = 4)
    private BigDecimal balance;            // 잔액

    private boolean locked;                // 잠긴 계좌면 거래 불가

    /** 입금: 0 이하는 거부 */
    public void deposit(BigDecimal amount) {
        if (amount == null || amount.signum() <= 0)
            throw new IllegalArgumentException("Deposit amount must be positive");
        this.balance = this.balance.add(amount);
    }

    /** 출금: 0 이하 거부 + 잔액 부족 거부 (음수 잔액 원천 차단) */
    public void withdraw(BigDecimal amount) {
        if (amount == null || amount.signum() <= 0)
            throw new IllegalArgumentException("Withdraw amount must be positive");
        if (this.balance.compareTo(amount) < 0)
            throw new IllegalStateException("Insufficient balance");
        this.balance = this.balance.subtract(amount);
    }
}
```

여기가 핵심입니다. balance를 바꾸는 방법이 **deposit과 withdraw 두 개뿐**입니다. setBalance() 같은 건 아예 없습니다.

> **왜?** 아무 데서나 account.setBalance(-5000)을 할 수 있으면 **음수 잔액이 생깁니다.** 규칙을 객체 안에 가두면 **어디서 호출하든 규칙이 깨지지 않습니다.**

TransactionLog (수신 원장)

```
@Entity @Table(name = "transaction_logs")
public class TransactionLog {
    @Enumerated(STRING) private TransactionType type;  // DEPOSIT/WITHDRAW/TRANSFER_IN/OUT
    @ManyToOne(fetch = LAZY) private Account fromAccount;  // 입금이면 null
    @ManyToOne(fetch = LAZY) private Account toAccount;    // 출금이면 null
    private BigDecimal amount;
    private BigDecimal balanceAfter;      // 거래 직후 잔액 ← 검산용
    private String description;
```

```
    private LocalDateTime createdAt;
}
```

수정 메서드가 하나도 없습니다. 만들 때 값을 넣고, 그 뒤론 읽기만 합니다(append-only).

LoanAccount (대출 계약) — 스냅샷의 실체

```
public static LoanAccount execute(LoanApplication application,
                                  Account linkedAccount, LocalDate executedAt) {
    LoanProduct product = application.getProduct();

    LoanAccount loan = new LoanAccount();
    loan.application  = application;
    loan.member       = application.getMember();
    loan.linkedAccount = linkedAccount;
    loan.principal    = application.getRequestedAmount();
    loan.balance      = application.getRequestedAmount();  // 처음엔 원금 전액이 미상환

    loan.interestRate      = product.getInterestRate();       // ← 스냅샷
    loan.overdueExtraRate  = product.getOverdueExtraRate();   // ← 스냅샷
    loan.prepaymentFeeRate = product.getPrepaymentFeeRate();  // ← 스냅샷
    loan.repaymentType     = product.getRepaymentType();      // ← 스냅샷

    loan.executedAt    = executedAt;
    loan.maturityDate  = executedAt.plusMonths(application.getTermMonths());
    return loan;
}
```

상품에서 값을 "복사"해 옵니다. 참조만 들고 있으면 상품이 바뀔 때 **과거 계약의 조건까지 같이 바뀝니다.**

```
/** 원금 상환: 잔액을 줄이고, 0이 되면 완납 처리 */
public void repayPrincipal(BigDecimal principalPortion) {
    if (this.status == PAID_OFF)
        throw new BusinessException(INVALID_STATE, "이미 완납된 대출입니다.");
    if (this.balance.compareTo(principalPortion) < 0)
        throw new BusinessException(INVALID_STATE, "상환 원금이 미상환 잔액을 초과합니다.");

    this.balance = this.balance.subtract(principalPortion);
    if (this.balance.signum() == 0) this.status = PAID_OFF;   // 자동 완납 전이
}
```

"잔액이 0이 되면 완납"이라는 규칙이 **객체 안에** 있습니다. 서비스가 깜빡해도 상태가 어긋나지 않습니다.

RepaymentSchedule / OverdueHistory — 상태를 바꾸는 메서드들

```
// RepaymentSchedule (상환 회차)
public void markPaid()           { ... }  // PENDING/OVERDUE → PAID
public void markOverdue()        { ... }  // PENDING → OVERDUE
public void cancelByPrepayment() { ... }  // 중도상환 시 남은 회차 → CANCELED (삭제 아님)
public boolean isOverdue()       { ... }

// OverdueHistory (연체 이력)
public static OverdueHistory open(RepaymentSchedule s, LocalDate start)  // 연체 시작
public void accrue(BigDecimal penalty, LocalDate accruedAt)              // 이자 가산 + 기산점 전진
public void resolve(LocalDate resolvedAt)                                // 연체 해소
```

```
public boolean alreadyAccruedOn(LocalDate date)                    // 오늘 이미 붙였나?
public boolean isResolved()
```

메서드 이름이 곧 업무 용어입니다. setStatus("PAID")가 아니라 markPaid()라고 쓰면, 코드를 읽는 사람이 **무슨 일이 일어나는지** 바로 압니다.

A-2. 계산 엔진 — 스프링을 모르는 순수 자바

이 부분은 **@Autowired도, JPA도 전혀 안 씁니다.** 숫자를 받아 숫자를 돌려줄 뿐입니다.

```
public interface RepaymentScheduleGenerator {
    /** 원금·연이율·개월수를 받아 회차별 (원금, 이자, 잔액) 목록을 만든다 */
    List<ScheduleLine> generate(BigDecimal principal, BigDecimal annualRate, int months);
}
```

구현체가 셋입니다.

클래스	방식
EqualPaymentGenerator	원리금균등 (매달 같은 금액)
EqualPrincipalGenerator	원금균등 (원금만 같게)
BulletGenerator	만기일시 (이자만 내다 마지막에 몰아서)

그리고 별도 계산기 셋:

클래스	계산식
AccruedInterestCalculator	잔여원금 × 금리 × 경과일/365
OverdueInterestCalculator	연체원리금 × min(약정+가산, **20%**) × 일수/365
PrepaymentFeeCalculator	잔여원금 × 수수료율 × 잔존일수/총일수

> **왜 순수 자바로 분리했나?** 1. **테스트가 빠르다** — 스프링을 띄우지 않으니 240조합을 몇 초 만에 돈다 2. **틀리면 바로 안다** — DB나 트랜잭션이 섞이면 "계산이 틀린 건지 저장이 틀린 건지" 헷갈린다 3. **재사용된다** — 실행에서도, 중도상환에서도, 배치에서도 같은 계산기를 쓴다

A-3. 리포지토리 — DB에 말 거는 층

```
// AccountRepository
List<Account> findByMember(Member member);
boolean existsByAccountNumber(String accountNumber);

@Query("select a.id from Account a where a.accountNumber = :accountNumber")
Optional<Long> findIdByAccountNumber(String n);   // ← ID만! (1차 캐시 오염 방지)

@Lock(LockModeType.PESSIMISTIC_WRITE)
Optional<Account> findWithLockingById(Long id);   // ← SELECT ... FOR UPDATE
```

```
// RepaymentScheduleRepository — 조건부 UPDATE와 커서 조회
@Modifying
@Query("""
   update RepaymentSchedule s set s.status = 'PAID', s.paidAt = :now
    where s.id = :id and s.status in ('PENDING','OVERDUE')
""")
int markPaidIfPayable(Long id, LocalDateTime now);   // 0이면 이미 누가 했다 → 409
```

```
@Query("""
   select s from RepaymentSchedule s
    where s.status = 'PENDING' and s.dueDate < :baseDate and s.id > :lastId
    order by s.id asc
""")
List<RepaymentSchedule> findOverdueCandidates(LocalDate baseDate, Long lastId, Pageable p);
```

두 번째 쿼리의 `s.id > :lastId`가 커서입니다. offset이 아니라 **마지막으로 읽은 id 다음부터** 읽습니다.

QueryDSL 커스텀 (동적 검색)

```
public Page<LoanAccount> search(Long memberId, LoanSearchCondition c, Pageable pageable) {
    List<LoanAccount> content = queryFactory
        .selectFrom(loanAccount)
        .join(loanAccount.application, loanApplication).fetchJoin()  // N+1 방지
        .join(loanApplication.product, loanProduct).fetchJoin()
        .where(
            loanAccount.member.id.eq(memberId),   // 항상 있는 조건
            statusEq(c.status()),                 // 없으면 null → 무시됨
            repaymentTypeEq(c.repaymentType()),
            productIdEq(c.productId()),
            executedBetween(c.executedFrom(), c.executedTo()),
            principalBetween(c.minPrincipal(), c.maxPrincipal())
        )
        .orderBy(loanAccount.id.desc())
        .offset(pageable.getOffset()).limit(pageable.getPageSize())
        .fetch();

    JPAQuery<Long> countQuery = ...;   // 개수는 따로
    return PageableExecutionUtils.getPage(content, pageable, countQuery::fetchOne);
}
```

`PageableExecutionUtils`는 "마지막 페이지면 COUNT 쿼리를 아예 안 날리는" 최적화를 해줍니다. 전체 개수를 이미 계산할 수 있으면 DB에 묻지 않습니다.

`fetchJoin()`은 N+1 문제를 막습니다.

> **N+1이 뭔가요?** 대출 10건을 조회했는데, 각 대출의 상품 이름을 꺼낼 때마다 쿼리가 한 번씩 더 나가서 **1 + 10 = 11번** 쿼리가 나가는 현상입니다. fetch join은 **처음부터 같이 가져와서 1번**으로 끝냅니다.

A-4. 서비스 — 규칙이 사는 곳

왜 서비스를 둘로 쪼갰나 (Service + Processor)

```
// LoanExecutionService  — 멱등키 껍데기 (트랜잭션 없음)
public LoanExecutionResponse execute(Long applicationId, String idemKey) {
    return idempotentExecutor.run(idemKey, memberId, requestHash,
            LoanExecutionResponse.class,
            () -> processor.execute(applicationId, idemKey));   // 실제 처리 위임
}

// LoanExecutionProcessor — 실제 처리 (@Transactional)
@Transactional
public LoanExecutionResponse execute(Long applicationId, String idemKey) { ... }
```

한 클래스 안에서 자기 메서드를 호출하면 스프링 프록시를 안 타서 @Transactional이 적용되지 않습니다. 멱등키 선점은 본 처리와 **다른 트랜잭션**이어야 하므로, "경계가 다른 두 책임"을 **두 개의 빈**으로 분리했습니다.

실행 처리의 전체 흐름

```
@Transactional
public LoanExecutionResponse execute(Long applicationId, String idemKey) {
    // 1) 조건부 상태 전이 — 0행이면 이미 누가 실행했다
    int updated = applicationRepository.markExecutedIfApproved(applicationId);
    if (updated == 0) throw new BusinessException(ALREADY_EXECUTED);

    // 2) 락 순서 규칙: Account 먼저
    Account account = accountRepository.findWithLockingById(linkedAccountId)...;

    // 3) 계약 생성 (조건 스냅샷)
    LoanAccount loan = LoanAccount.execute(application, account, LocalDate.now());
    loanAccountRepository.save(loan);

    // 4) 상환 스케줄 전 회차 생성
    generator.generate(principal, rate, months)
             .forEach(line -> scheduleRepository.save(RepaymentSchedule.of(...)));

    // 5) 연결계좌에 원금 입금 + 수신 원장
    account.deposit(principal);
    transactionLogRepository.save(TransactionLog.deposit(account, principal));

    // 6) 대출 원장
    loanTransactionRepository.save(LoanTransaction.execution(loan, principal, idemKey));

    // 7) 멱등키에 응답 저장 (같은 트랜잭션 — MANDATORY)
    idempotentExecutor.complete(idemKey, response);
    return response;
}
```

이 7단계가 전부 하나의 트랜잭션입니다. 중간에 하나라도 실패하면 **전부 없던 일**이 됩니다.

상환에서 락 순서를 지키는 방법

```
// 대출을 "엔티티로" 읽으면 1차 캐시가 오염된다 → ID만 프로젝션으로
Long linkedAccountId = loanAccountRepository.findLinkedAccountIdById(loanId)...;

// 규칙: Account → LoanAccount 순서로 잠근다
Account account   = accountRepository.findWithLockingById(linkedAccountId)...;
LoanAccount loan  = loanAccountRepository.findWithLockingById(loanId)...;
```

자동심사 — 룰을 순서대로 돌린다

```
private void review(LoanApplication application) {
    ReviewContext context = new ReviewContext(member, product, amount, term);

    for (LoanReviewer rule : reviewers) {         // 스프링이 @Order 순서로 주입
        Optional<String> rejectCode = rule.reject(context);
        if (rejectCode.isPresent()) {
            application.reject(rejectCode.get()); // 첫 탈락 사유를 기록
```

```
            return;
        }
    }
    application.approve();
}
```

룰을 추가하려면 클래스 하나만 더 만들면 됩니다. 이 코드는 손대지 않습니다(OCP — 확장에 열려 있고 변경에 닫혀 있다).

A-5. 멱등 실행기 — 전체 코드

```
public <T> T run(String idemKey, Long memberId, String requestHash,
                 Class<T> responseType, Supplier<T> action) {
    validateKey(idemKey);

    Optional<String> replay;
    try {
        // 선점 시도 (REQUIRES_NEW — 별도 트랜잭션)
        replay = store.claim(idemKey, memberId, requestHash);
    } catch (DataIntegrityViolationException e) {
        // UNIQUE 위반 = 동시에 다른 요청이 선점 성공 → 그 결과를 읽어서 돌려준다
        return deserialize(store.findCompletedResponse(idemKey, requestHash), responseType);
    }

    if (replay.isPresent()) {
        return deserialize(replay.get(), responseType);   // 이미 완료된 요청 → 응답 재생
    }

    try {
        return action.get();          // 본 처리 (여기서 @Transactional 시작)
    } catch (RuntimeException e) {
        store.release(idemKey);       // 실패하면 키 해제 (REQUIRES_NEW)
        throw e;
    }
}
```

주의 깊게 볼 점: `claim`에서 터진 중복키 예외를 **일부러 안 잡고** 밖으로 흘립니다. 그 트랜잭션 안에서 잡으면 이미 오염된 트랜잭션이라 이후 쿼리가 전부 실패합니다. 그래서 **바깥에서 새 트랜잭션으로 다시 읽습니다.**

A-6. 컨트롤러 — 엔드포인트 전체 목록

컨트롤러는 **얇게** 유지합니다. 검증(`@Valid`)만 걸고 실제 로직은 서비스에 넘깁니다.

인증 `/api/auth`

메서드	경로	하는 일
POST	/register	가입 + 토큰 발급
POST	/login	로그인 + 토큰 발급
POST	/refresh	토큰 재발급 (Rotation)
POST	/logout	토큰 폐기

계좌 `/api/accounts` (JWT 필요)

메서드	경로	하는 일
POST	/	계좌 개설 → 201 + Location
GET	/	내 계좌 목록
GET	/{id}	계좌 상세 (소유권 검증)
GET	/{id}/transactions	거래내역 (페이징)
POST	/{id}/deposit	입금
POST	/{id}/withdraw	출금
POST	/transfer	이체

대출 상품 /api/loans/products

메서드	경로	하는 일
GET	/	판매 중 상품 목록

대출 신청 /api/loans/applications

메서드	경로	하는 일
POST	/	신청 + **즉시 자동심사**
GET	/	내 신청 이력
GET	/{id}	신청 상세
DELETE	/{id}	신청 취소

대출 계약 /api/loans

메서드	경로	헤더	하는 일
POST	/applications/{applicationId}/execute	**Idempotency-Key**	실행
POST	/{loanId}/repayments	**Idempotency-Key**	회차 상환
POST	/{loanId}/prepayment	**Idempotency-Key**	전액 중도상환
GET	/		내 대출 목록
GET	/search		조건 검색 (QueryDSL)
GET	/{loanId}		대출 상세
GET	/{loanId}/schedules		상환 스케줄

돈이 움직이는 세 API에만 Idempotency-Key 헤더가 필수입니다. 없으면 400입니다.

A-7. 프론트엔드에서 멱등키 다루기

```
// 사용자 행동 1회 = 키 1개
const key = crypto.randomUUID();

async function repay() {
  try {
    return await idempotentPost(`/loans/${loanId}/repayments`, {
      key,                          // ← 같은 키를 유지
      body: { installmentNo },
      retryOnNetworkFailure: true   // 네트워크 실패 시 같은 키로 재시도
    });
  } catch (e) {
```

```
    if (e.status === 409) showMessage('이미 처리되었거나 처리 중입니다');
    if (e.status === 422) showMessage('요청이 변경되었습니다. 새로고침 후 다시 시도하세요');
  }
}
```

ApiError에 **HTTP 상태와 서버 에러 코드를 같이 실어서** 화면이 상황별로 다른 안내를 할 수 있게 했습니다.

부록 B. 용어 사전

용어	쉬운 뜻
엔티티(Entity)	DB 표의 한 줄에 대응하는 자바 객체
트랜잭션	“전부 되거나, 전부 안 되거나”로 묶는 단위
커밋 / 롤백	확정 / 없던 일로 되돌리기
비관적 락	“내가 쓸 동안 아무도 손대지 마” — 미리 잠그기
낙관적 락	“일단 하고, 충돌났으면 다시 하자” — 버전으로 검사
데드락	서로 상대를 기다리다 영원히 멈춤
stale read	낡은 값을 읽는 것
1차 캐시	같은 트랜잭션에서 읽은 걸 기억해 두는 JPA의 임시 저장소
격리수준	동시에 도는 트랜잭션끼리 서로를 얼마나 볼 수 있는지
REPEATABLE READ	MySQL 기본값. 트랜잭션 시작 시점의 “사진”을 계속 봄
멱등(idempotent)	여러 번 해도 결과가 한 번 한 것과 같음
UNIQUE 제약	같은 값이 두 번 들어올 수 없게 하는 DB 규칙
인덱스	빨리 찾기 위한 색인(책 뒤 찾아보기)
풀스캔	표를 처음부터 끝까지 다 훑는 것
EXPLAIN	“이 쿼리를 어떻게 실행할 건지” DB에게 물어보는 명령
N+1 문제	목록 1번 + 각 항목마다 1번씩 쿼리가 더 나가는 현상
fetch join	연관 데이터를 처음부터 같이 가져와 N+1을 막는 것
청크(chunk)	배치에서 한 트랜잭션에 묶는 건수
커서 페이징	위치(offset)가 아니라 “마지막 id 다음부터” 읽는 방식
마이그레이션	DB 표 변경을 번호 붙인 파일로 관리하는 것
스냅샷	그 시점의 값을 복사해 보관한 것
원장(ledger)	모든 거래를 순서대로 쌓아둔, 지우지 않는 기록
DTO	계층 사이를 오가는 데이터 그릇
프록시	스프링이 `@Transactional` 같은 걸 걸기 위해 감싸 만든 대역 객체
OCP	확장에는 열려 있고 변경에는 닫혀 있는 설계 원칙
CI	코드를 올릴 때마다 자동으로 빌드·테스트하는 것